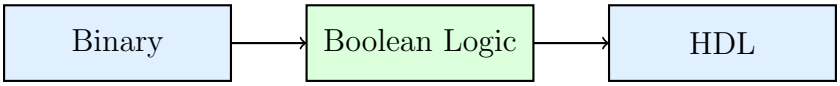
Digital Logic Design
Quick Revision Notes

Number Systems, Gates, Boolean Algebra, HDL, Adders and Logic Families



Compact Exam Revision Material

Ideal for Quick Revision and Viva Preparation

Contents

1 Number Systems **3**
1.1 Decimal Number System . . . 3
1.2 Binary Number System . . . 3
1.3 Hexadecimal Number System . . . 3
1.4 Decimal to Binary Conversion . . . 3
1.5 Binary to Decimal Conversion . . . 4
1.6 Hexadecimal to Decimal . . . 4
1.7 Two's Complement . . . 4

2 Gate Theory **4**
2.1 AND Gate . . . 4
2.2 OR Gate . . . 5
2.3 NOT Gate . . . 5

3 NAND Gate Universality **5**
3.1 NOT using NAND . . . 5
3.2 AND using NAND . . . 5
3.3 OR using NAND . . . 5

4 Boolean Algebra **5**
4.1 Identity Laws . . . 5
4.2 Complement Laws . . . 6
4.3 De Morgan's Laws . . . 6
4.4 Simplification Example . . . 6

5 Bitwise Operations in Programming **6**
5.1 Bitwise AND . . . 6
5.2 Bitwise OR . . . 6
5.3 Bitwise XOR . . . 6
5.4 Bitwise NOT . . . 6
5.5 Python Example . . . 7

6 HDL Syntax (Nand2Tetris) **7**
6.1 Basic Structure . . . 7
6.2 Hardware Simulator Workflow . . . 7

7 Logic Devices **7**
7.1 XOR Gate . . . 7
7.2 Multiplexer (Mux) . . . 8
7.3 Demultiplexer (DMux) . . . 8

8 Adders **8**
8.1 Half Adder . . . 8
8.2 Full Adder . . . 8
8.3 Ripple Carry Adder . . . 8

9 Logic Families **8**
9.1 TTL 8
9.2 CMOS 9
9.3 TTL vs CMOS 9
9.4 Gate Delay 9
9.5 Power Consumption 9

10 Important Formula Sheet **9**

11 Quick Viva Questions **10**

12 Exam Tips **10**

13 Summary **10**

1. Number Systems

1.1. Decimal Number System

$$(572)_{10}$$

Base:

$$10$$

Digits:

$$0 - 9$$

1.2. Binary Number System

$$(101101)_2$$

Base:

$$2$$

Digits:

$$0, 1$$

1.3. Hexadecimal Number System

$$(2AF)_{16}$$

Base:

$$16$$

Digits:

$$0 - 9,\ A - F$$

1.4. Decimal to Binary Conversion

Convert:

$$13_{10}$$

$$13 \div 2 = 6\ remainder\ 1$$

$$6 \div 2 = 3\ remainder\ 0$$

$$3 \div 2 = 1\ remainder\ 1$$

$$1 \div 2 = 0\ remainder\ 1$$

Answer:

$$1101_2$$

1.5. Binary to Decimal Conversion

Convert:

$$1011_2$$

$$1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0$$

$$8 + 0 + 2 + 1 = 11$$

1.6. Hexadecimal to Decimal

$$2AF_{16}$$

$$2 \times 16^2 + 10 \times 16^1 + 15 \times 16^0$$

$$512 + 160 + 15 = 687$$

1.7. Two's Complement

Steps:

1. Invert all bits
2. Add 1

Example:

$$0101$$

Invert:

$$1010$$

Add 1:

$$1011$$

Thus:

$$-5 = 1011$$

2. Gate Theory

2.1. AND Gate

$$Y = A \cdot B$$

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

2.2. OR Gate

$$Y = A + B$$

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

2.3. NOT Gate

$$Y = \overline{A}$$

A	Y
0	1
1	0

3. NAND Gate Universality

NAND is called a:

Universal Gate

because all logic gates can be implemented using only NAND gates.

3.1. NOT using NAND

$$Y = A \; NAND \; A$$

3.2. AND using NAND

$$A \cdot B = (A \; NAND \; B) \; NAND \; (A \; NAND \; B)$$

3.3. OR using NAND

Using De Morgan's Law:

$$A + B = (\overline{A} \cdot \overline{B})'$$

4. Boolean Algebra

4.1. Identity Laws

$$A + 0 = A$$

$$A \cdot 1 = A$$

4.2. Complement Laws

$$A + \overline{A} = 1$$

$$A \cdot \overline{A} = 0$$

4.3. De Morgan's Laws

$$\overline{A + B} = \overline{A} \cdot \overline{B}$$

$$\overline{AB} = \overline{A} + \overline{B}$$

4.4. Simplification Example

Simplify:

$$A + AB$$

Using absorption law:

$$A + AB = A$$

5. Bitwise Operations in Programming

5.1. Bitwise AND

$$5 \ \& \ 3$$

$$0101 \& 0011 = 0001$$

5.2. Bitwise OR

$$5 \mid 3$$

$$0101 | 0011 = 0111$$

5.3. Bitwise XOR

$$5 \hat{} 3$$

$$0101 \oplus 0011 = 0110$$

5.4. Bitwise NOT

$$\sim 0101 = 1010$$

5.5. Python Example

```
a = 5
b = 3

print(a & b)
print(a | b)
print(a ^ b)
```

6. HDL Syntax (Nand2Tetris)

6.1. Basic Structure

```
CHIP And {

    IN a, b;

    OUT out;

    PARTS:
    Nand(a=a, b=b, out=n1);

    Nand(a=n1, b=n1, out=out);
}
```

6.2. Hardware Simulator Workflow

1. Write HDL code
2. Load chip in simulator
3. Run test script
4. Compare output

7. Logic Devices

7.1. XOR Gate

$$A \oplus B$$

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

7.2. Multiplexer (Mux)

Selects one input among many.

$$Y = \overline{S}A + SB$$

7.3. Demultiplexer (DMux)

Routes one input to multiple outputs.

8. Adders

8.1. Half Adder

Outputs:

$$Sum = A \oplus B$$

$$Carry = A \cdot B$$

8.2. Full Adder

$$Sum = A \oplus B \oplus C_{in}$$

$$C_{out} = AB + BC_{in} + AC_{in}$$

8.3. Ripple Carry Adder

A chain of Full Adders.

Carry propagates stage by stage.

$$Delay = n \times T_{FA}$$

9. Logic Families

9.1. TTL

TTL:

Transistor-Transistor Logic

Uses:

$$BJT$$

Features:

- Fast
- Higher power consumption

9.2. CMOS

CMOS:

Complementary MOS

Uses:

$$MOSFET$$

Features:

- Very low power
- High integration density

9.3. TTL vs CMOS

Feature	TTL	CMOS
Technology	BJT	MOSFET
Power	High	Low
Speed	Fast	Moderate/Fast
Heat	More	Less

9.4. Gate Delay

Propagation delay:

$$t_{pd}$$

Time taken for output to respond after input change.

9.5. Power Consumption

$$P = VI$$

For CMOS:

$$P = \alpha C V^2 f$$

10. Important Formula Sheet

$$A + \overline{A} = 1$$

$$A \cdot \overline{A} = 0$$

$$Sum = A \oplus B$$

$$Carry = A \cdot B$$

$$t_{pd} = t_2 - t_1$$

$$P = VI$$

11. Quick Viva Questions

1. Why is NAND called universal gate?
2. Difference between TTL and CMOS?
3. What is propagation delay?
4. What is two's complement?
5. Difference between Half Adder and Full Adder?
6. Why is CMOS preferred in laptops?
7. What is the purpose of Mux?
8. What is De Morgan's Law?

12. Exam Tips

Exam Tip

Remember all truth tables carefully. Most numerical and circuit design questions depend on them.

Exam Tip

Practice binary arithmetic and two's complement regularly.

Exam Tip

Always verify carry propagation in Ripple-Carry Adders.

13. Summary

This revision material covered:

- Number systems
- Logic gates
- NAND universality
- Boolean algebra
- Bitwise operations
- HDL basics
- Multiplexers and adders
- Logic families

These concepts form the foundation of Digital Logic Design and Computer Architecture.